```
                    idx += 1
                    continue

                callsign_hint = None
                if idx + 1 < len(clean_lines):
                    ascii_candidate = clean_lines[idx + 1].upper()
                    if not hex_line_re.match(ascii_candidate):
                        tokens = [t.strip() for t in ascii_candidate.split(",") if t.strip()]
                        if tokens and tokens[0] != "NONE":
                            callsign_hint = tokens[0]
                        idx += 1

                timestamp = (base_time + timedelta(seconds=event_idx)).isoformat() + "Z"
                rows.append((timestamp, "", payload, callsign_hint))
                event_idx += 1
                idx += 1

        # Split-file format support:
        # - Hex file: <date_time>_engi_H.txt (one hex payload per line)
        # - Ascii file: matching <date_time>_engi_A.txt (one ascii line per line)
        if not rows:
            hex_line_re = re.compile(r"^[0-9a-fA-F]{10,}$")
            hex_lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
            if hex_lines and all(hex_line_re.match(ln) for ln in hex_lines):
                ascii_lines = []
                ascii_candidates = []
                stem = path.stem
                if stem.endswith("_engi_H"):
                    ascii_candidates.append(path.with_name(path.name.replace("_engi_H.txt", "_engi_A.txt")))
                    if path.parent.name.lower() == "hex_data":
                        ascii_candidates.append(path.parent.parent / "ascii_data" / path.name.replace("_engi_H.txt",
    | "_engi_A.txt"))
                ascii_candidates.append(path.with_name(f"{stem}_A.txt"))
                ascii_candidates.append(path.with_name(f"{stem}_ascii.txt"))

                for ascii_path in ascii_candidates:
                    if ascii_path.exists():
                        ascii_lines = [
                            ln.strip()
                            for ln in ascii_path.read_text(encoding="utf-8", errors="ignore").splitlines()
                            if ln.strip()
                        ]
                        break

                base_time = datetime.utcnow()
                for idx, hex_data in enumerate(hex_lines):
                    try:
                        payload = bytes.fromhex(hex_data)
                    except ValueError:
                        continue

                    callsign_hint = None
                    if idx < len(ascii_lines):
                        ascii_candidate = ascii_lines[idx].upper()
                        tokens = [t.strip() for t in ascii_candidate.split(",") if t.strip()]
                        if tokens and tokens[0] != "NONE":
                            callsign_hint = tokens[0]

                    timestamp = (base_time + timedelta(seconds=idx)).isoformat() + "Z"
                    rows.append((timestamp, "", payload, callsign_hint))

        if max_entries is not None and max_entries > 0:
            rows = rows[-max_entries:]

        return rows

    def get_state_snapshot(self, stale_seconds=45):
        now = time.time()
        rows = []
        stale_keys = []

        for key, value in self._state.items():
            last_seen = value.get("_last_seen", now)
```